# Complete Spring and Spring Boot
# Made Simple

Every topic explained with a picture you will never forget
+ a working code example

## How to use these notes

1. Read the **story** - it is the picture that sticks.
2. Type the **code** in a project from start.spring.io. Never copy-paste.
3. Say the **REMEMBER** line out loud.
4. Next day, rebuild the example from memory.

## Before Spring, know these Java tools

Classes and objects, interfaces, inheritance, collections (List, Map), generics, exceptions, streams and lambdas, and basic SQL. Install JDK 17 or newer, Maven and an IDE (IntelliJ IDEA or Eclipse STS). Code here uses jakarta imports (Spring Boot 3 and newer); always create new projects from start.spring.io to get the latest stable version.

# Contents


## PART 1 - SPRING FRAMEWORK CORE

1. What are Spring and Spring Boot? | 2. Inversion of Control (IoC) and Dependency Injection (DI) | 3. Beans and the Spring Container | 4. Creating Beans: @Component, @Service, @Repository, @Bean | 5. Types of Injection: Constructor, Setter, Field | 6. Bean Scopes | 7. Bean Lifecycle | 8. Properties, @Value and Profiles | 9. AOP: Aspect Oriented Programming | 10. Application Events

## PART 2 - SPRING BOOT ESSENTIALS

11. Creating a Spring Boot Project | 12. Starters and Auto-Configuration | 13. application.properties vs YAML | 14. Actuator and DevTools | 15. Logging

## PART 3 - BUILDING REST APIs WITH SPRING MVC

16. How a Web Request Travels (Spring MVC) | 17. @RestController and HTTP Mappings | 18. Reading Input and ResponseEntity | 19. DTOs and Records | 20. Validation | 21. Exception Handling | 22. Layered Architecture | 23. CORS and API Documentation (Swagger) | 24. Calling Other APIs: RestClient and WebClient

## PART 4 - DATABASES WITH SPRING DATA JPA

25. JPA, Hibernate and Entities | 26. Spring Data JPA Repositories | 27. Derived Queries, @Query, Paging and Sorting | 28. Entity Relationships | 29. Transactions: @Transactional | 30. Databases, Schema and Migrations | 31. The N+1 Problem and Lazy Loading

## PART 5 - SECURITY, TESTING AND PRODUCTION

32. Spring Security Basics | 33. Users from the Database: UserDetailsService | 34. JWT Authentication Flow | 35. JWT Filter | 36. Testing Spring Boot Apps | 37. Caching | 38. Scheduling and Async | 39. Microservices and Spring Cloud | 40. Messaging with Kafka (and RabbitMQ) | 41. Docker and Deployment | 42. Best Practices | 43. Common Mistakes to Avoid

## FINAL

Project: To-Do App | One-page Cheat Sheet | Learning Roadmap

# PART 1 - SPRING FRAMEWORK CORE

## 1. What are Spring and Spring Boot?

**Spring** is a powerful Java framework that manages your objects and wires them together, like a **building contractor** who hires electricians, plumbers and painters so you only design the house. **Spring Boot** is Spring with **everything pre-assembled**: embedded server, automatic setup, no XML. Spring = engine parts. Spring Boot = the finished car, ready to drive.

```
// The ENTIRE Spring Boot web app can be this small:
@SpringBootApplication
public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}

@RestController
class HelloController {
    @GetMapping("/hello")
    String hello() {
        return "Hello, Spring Boot!";    // open http://localhost:8080/hello
    }
}
// Needs: Java 17+, Maven or Gradle. No Tomcat install, no XML.
```

**REMEMBER:** Spring = manages objects. Spring Boot = Spring that starts with zero setup.

## 2. Inversion of Control (IoC) and Dependency Injection (DI)

Normally YOU create the objects you need (**new**). With IoC, **Spring creates them and hands them to you**. Restaurant story: you do not go into the kitchen and cook - the waiter brings the dish. A **dependency** is something a class needs (a chef needs a knife). **Injection** = Spring gives it, you don't build it.

```
// WITHOUT Spring: tightly glued, hard to test or change
class OrderService {
    private EmailService email = new EmailService();   // you create it
}

// WITH Spring: loosely coupled
@Service
class EmailService { void send(String to) { /* ... */ } }

@Service
class OrderService {
    private final EmailService email;

    OrderService(EmailService email) {     // Spring injects it for you
        this.email = email;
    }
    void placeOrder() { email.send("customer@mail.com"); }
}
// Benefits: easy to swap implementations, easy to test with fake objects.
```

**REMEMBER:** IoC = Spring controls object creation. DI = Spring hands you what you need.

## 3. Beans and the Spring Container

A **bean** is simply an object that Spring manages. The **container** (ApplicationContext) is the warehouse that creates, stores and delivers beans. Spring finds beans by scanning your code for special annotations (component scanning).

```
@SpringBootApplication
public class App {
    public static void main(String[] args) {
        // run() returns the container
        ApplicationContext ctx = SpringApplication.run(App.class, args);

        OrderService svc = ctx.getBean(OrderService.class);   // ask for a bean
        svc.placeOrder();

        System.out.println(ctx.getBeanDefinitionCount() + " beans loaded");
    }
}
// Container types: BeanFactory (basic)  ->  ApplicationContext (full-featured).
// In real code you rarely call getBean(); you just use dependency injection.
```

**REMEMBER:** Bean = object managed by Spring. ApplicationContext = the bean warehouse.

## 4. Creating Beans: @Component, @Service, @Repository, @Bean

Stereotype annotations are **name badges** that tell Spring 'manage me'. They all work the same; the badge shows the role. Use **@Bean** inside a @Configuration class when you cannot annotate the class (third-party libraries).

```
@Component   class Helper { }              // general-purpose bean
@Service     class UserService { }          // business logic
@Repository  class UserDao { }              // database access (adds exception translation)
@Controller  class PageController { }       // web layer (returns views)
@RestController class ApiController { }     // web layer (returns JSON)

// @Bean: you build the object, Spring manages it
@Configuration
class AppConfig {
    @Bean
    public ModelMapper modelMapper() {
        return new ModelMapper();           // library class, not ours
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}
```

**REMEMBER:** @Component family = my classes. @Bean method = classes I cannot annotate.

## 5. Types of Injection: Constructor, Setter, Field

Three ways to receive a dependency. **Constructor injection is the best**: the object is never half-built, fields can be final, and tests are easy. Field injection hides dependencies and is discouraged.

```
@Service
class A {
    private final B b;
    A(B b) { this.b = b; }                  // 1. CONSTRUCTOR (recommended)
    // With ONE constructor, @Autowired is optional.
}

@Service
class C {
    private B b;
    @Autowired
    void setB(B b) { this.b = b; }          // 2. SETTER (optional dependencies)
}

@Service
class D {
    @Autowired private B b;                 // 3. FIELD (avoid)
}

// Two beans of the same type? Tell Spring which one:
interface Payment { }
@Service @Primary   class UpiPayment implements Payment { }   // default choice
@Service("card")    class CardPayment implements Payment { }

@Service
class Checkout {
    Checkout(@Qualifier("card") Payment p) { }                 // choose by name
}
```

**REMEMBER:** Prefer constructor injection. @Primary = default, @Qualifier = pick by name.

## 6. Bean Scopes

Scope = **how many copies** Spring makes. **singleton** (default) = one shared copy for the whole app, like one school principal. **prototype** = a new copy every time asked, like handing each student a fresh notebook. Web scopes: **request** and **session**.

```
@Component                          // default = singleton
class Shared { }

@Component
@Scope("prototype")                 // new object on every injection/getBean
class Fresh { }

@Component
@Scope(value = WebApplicationContext.SCOPE_REQUEST,
       proxyMode = ScopedProxyMode.TARGET_CLASS)
class RequestInfo { }               // one per HTTP request

// Because singletons are shared by all users at once:
//   NEVER store user-specific data in fields of a singleton bean!
//   (Thread-safety trap: keep services stateless.)
```

**REMEMBER:** Singleton = one shared bean (default). Keep singletons stateless.

## 7. Bean Lifecycle

Beans have a life: **born (constructor) -> wired (dependencies injected) -> init -> in use -> destroy**. You can run code at the init and destroy moments.

```
@Component
public class CacheLoader {

    public CacheLoader() { System.out.println("1. Constructor"); }

    @PostConstruct                          // jakarta.annotation.PostConstruct
    void init() { System.out.println("2. Ready: load cache"); }

    @PreDestroy                             // jakarta.annotation.PreDestroy
    void cleanup() { System.out.println("3. Closing: free resources"); }
}

// Same idea with @Bean:
// @Bean(initMethod = "start", destroyMethod = "stop")

// Order: constructor -> dependency injection -> @PostConstruct
//        -> bean used -> @PreDestroy -> bean removed
```

**REMEMBER:** Constructor, inject, @PostConstruct, use, @PreDestroy.

## 8. Properties, @Value and Profiles

Never hard-code settings. Put them in **application.properties** and read them. **Profiles** give different settings for dev, test and production, like switching clothes for different weather.

```
# application.properties
app.name=Student Portal
app.max-users=100
server.port=8081

# application-dev.properties  (used when profile = dev)
spring.datasource.url=jdbc:h2:mem:devdb

# application-prod.properties
spring.datasource.url=jdbc:mysql://prod-host:3306/school

// Reading values
@Component
class Info {
    @Value("${app.name}") private String name;
    @Value("${app.max-users:50}") private int max;     // :50 = default value
}

// Group many values (better)
@ConfigurationProperties(prefix = "app")
record AppProps(String name, int maxUsers) { }         // add @EnableConfigurationProperties

// Activate a profile:  spring.profiles.active=dev
// A bean for one profile only:  @Profile("dev") @Component class DevSeeder {}
```

**REMEMBER:** @Value for one setting, @ConfigurationProperties for groups, profiles per environment.

## 9. AOP: Aspect Oriented Programming

Some jobs (logging, timing, security) are needed in **many places**. Instead of copy-pasting, write them once and have Spring **wrap** your methods, like a security guard at every door of a building. Terms: **Aspect** (the guard's duty), **Advice** (what to do), **Pointcut** (which doors).

```
// dependency: spring-boot-starter-aop
@Aspect
@Component
public class LoggingAspect {

    // Pointcut: every method in the service package
    @Before("execution(* com.example.service.*.*(..))")
    public void logBefore(JoinPoint jp) {
        System.out.println("Calling: " + jp.getSignature().getName());
    }

    @Around("execution(* com.example.service.*.*(..))")
    public Object time(ProceedingJoinPoint pjp) throws Throwable {
        long start = System.currentTimeMillis();
        Object result = pjp.proceed();                // run the real method
        System.out.println(pjp.getSignature() + " took "
            + (System.currentTimeMillis() - start) + " ms");
        return result;
    }
}
// Advice types: @Before @After @AfterReturning @AfterThrowing @Around
// @Transactional and Spring Security use AOP behind the scenes.
```

**REMEMBER:** AOP = write cross-cutting code once, apply it to many methods.

## 10. Application Events

A **school announcement**: one part of the app shouts 'something happened!' and any interested part listens, without the two knowing each other.

```
record UserRegistered(String email) { }          // the event

@Service
class UserService {
    private final ApplicationEventPublisher publisher;
    UserService(ApplicationEventPublisher publisher) { this.publisher = publisher; }

    void register(String email) {
        // ... save user ...
        publisher.publishEvent(new UserRegistered(email));   // shout
    }
}

@Component
class WelcomeEmailListener {
    @EventListener
    void on(UserRegistered e) {                              // listen
        System.out.println("Sending welcome mail to " + e.email());
    }
}
// Add @Async to the listener to run it in the background.
```

**REMEMBER:** publishEvent() shouts, @EventListener hears. Loose coupling.

# PART 2 - SPRING BOOT ESSENTIALS

## 11. Creating a Spring Boot Project

Go to **start.spring.io** (Spring Initializr), pick Maven/Gradle, Java 17+, add dependencies, download the zip, open in IntelliJ or Eclipse. Always use the latest stable Boot version offered there.

```
demo/
 +-- pom.xml                    dependencies and build (Maven)
 +-- src/main/java/com/example/demo/
 |     +-- DemoApplication.java     main class with @SpringBootApplication
 |     +-- controller/  service/  repository/  model/
 +-- src/main/resources/
 |     +-- application.properties   settings
 |     +-- static/  templates/      web files
 +-- src/test/java/...              tests

// @SpringBootApplication = 3 annotations in one:
//   @Configuration            this class can define beans
//   @EnableAutoConfiguration  set things up automatically
//   @ComponentScan            find my @Component classes in this package and below

// Run:  ./mvnw spring-boot:run     or press Run in the IDE
// Build: ./mvnw clean package   -> target/demo-0.0.1-SNAPSHOT.jar
// Start: java -jar target/demo-0.0.1-SNAPSHOT.jar
```

**REMEMBER:** Keep the main class in the root package so scanning finds everything below.

## 12. Starters and Auto-Configuration

A **starter** is a **combo meal**: add one dependency and get everything related, with compatible versions. **Auto-configuration** is the waiter who sees the ingredients in your kitchen and prepares the dish for you. Add the web starter and Tomcat plus JSON converters appear automatically.

```
<!-- pom.xml -->
<dependencies>
  <dependency>   <!-- REST APIs: Spring MVC + Tomcat + Jackson -->
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>   <!-- Database with JPA / Hibernate -->
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>   <!-- Login and security -->
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
  </dependency>
  <dependency>   <!-- Input validation -->
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
  </dependency>
  <dependency>   <!-- Tests -->
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>
<!-- See what was auto-configured:  debug=true  in application.properties -->
```

**REMEMBER:** Starter = one dependency, whole feature. Auto-config = Boot guesses the setup from the classpath.

## 13. application.properties vs YAML

Two formats for the same settings. **YAML** is tidier for nested values.

```
# application.properties
server.port=8080
spring.datasource.url=jdbc:mysql://localhost:3306/school
spring.datasource.username=root
spring.datasource.password=${DB_PASSWORD}
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

# application.yml  (same thing)
server:
  port: 8080
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/school
    username: root
    password: ${DB_PASSWORD}          # from an environment variable
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true

# Priority (high to low): command line > environment variables
#   > application-{profile}.yml > application.yml
# NEVER commit real passwords. Use environment variables or a secrets manager.
```

**REMEMBER:** Same settings, two formats. Secrets come from environment variables.

## 14. Actuator and DevTools

**DevTools** restarts your app automatically when code changes (a self-refreshing kitchen). **Actuator** is the **health dashboard** of a car: health, metrics, environment, beans.

```
<!-- pom.xml -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-devtools</artifactId>
  <scope>runtime</scope>
  <optional>true</optional>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

# application.properties
management.endpoints.web.exposure.include=health,info,metrics
management.endpoint.health.show-details=always

# Try in the browser:
#   /actuator/health     -> {"status":"UP"}
#   /actuator/metrics    -> list of metrics
#   /actuator/info       -> app info
# Never expose every endpoint publicly; secure them in production.
```

**REMEMBER:** DevTools = auto-restart. Actuator = /actuator/health and friends.

## 15. Logging

Use a **logger** instead of System.out.println. Levels from quiet to loud: **ERROR, WARN, INFO, DEBUG, TRACE**. Boot uses SLF4J + Logback automatically.

```
@Service
public class PaymentService {
    private static final Logger log = LoggerFactory.getLogger(PaymentService.class);

    void pay(int id) {
        log.info("Payment started for order {}", id);       // {} placeholder
        try {
            // ...
            log.debug("Gateway responded OK");
        } catch (Exception e) {
            log.error("Payment failed for order {}", id, e);   // prints stack trace
        }
    }
}
// With Lombok:  @Slf4j on the class, then use  log.info("...")

# application.properties
logging.level.root=INFO
logging.level.com.example=DEBUG
logging.level.org.hibernate.SQL=DEBUG
logging.file.name=app.log
```

**REMEMBER:** log.info("text {}", value). Never log passwords.

# PART 3 - BUILDING REST APIs WITH SPRING MVC

## 16. How a Web Request Travels (Spring MVC)

Think of a **restaurant**: the **DispatcherServlet** is the receptionist who receives every guest, the **HandlerMapping** finds the right waiter (controller), the controller asks the kitchen (service), and the answer goes back converted to JSON.

```
Browser / Postman
      |  HTTP request:  GET /api/students/5
      v
DispatcherServlet            (front controller - the receptionist)
      |  finds the right method via HandlerMapping
      v
@RestController method       (the waiter)
      |  calls
      v
@Service                     (business rules - the chef)
      |  calls
      v
@Repository / Database       (the pantry)
      |
      v   result object  -> Jackson converts it to JSON
HTTP response:  200 OK   {"id":5,"name":"Asha"}

// MVC = Model (data) + View (display) + Controller (traffic cop)
// In REST APIs the "view" is just JSON.
```

**REMEMBER:** One front door (DispatcherServlet) sends each request to the right controller method.

## 17. @RestController and HTTP Mappings

A controller is the **waiter** who takes requests. Each HTTP verb has a meaning: **GET** read, **POST** create, **PUT** replace, **PATCH** partly update, **DELETE** remove.

```
@RestController                      // = @Controller + @ResponseBody (returns JSON)
@RequestMapping("/api/students")     // base URL for the whole class
public class StudentController {

    private final StudentService service;
    public StudentController(StudentService service) { this.service = service; }

    @GetMapping                      // GET /api/students
    public List<Student> all() { return service.findAll(); }

    @GetMapping("/{id}")             // GET /api/students/5
    public Student one(@PathVariable Long id) { return service.findById(id); }

    @PostMapping                     // POST /api/students
    public Student create(@RequestBody Student s) { return service.save(s); }

    @PutMapping("/{id}")             // PUT /api/students/5
    public Student update(@PathVariable Long id, @RequestBody Student s) {
        return service.update(id, s);
    }

    @DeleteMapping("/{id}")          // DELETE /api/students/5
    public void delete(@PathVariable Long id) { service.delete(id); }
}
```

**REMEMBER:** @RestController + @RequestMapping + @GetMapping/@PostMapping/@PutMapping/@DeleteMapping.

## 18. Reading Input and ResponseEntity

Data arrives in three places: the **URL path** (@PathVariable), the **query string** (@RequestParam), and the **body** (@RequestBody). **ResponseEntity** lets you control the status code and headers of the answer.

```
// GET /api/students/5                    -> @PathVariable
// GET /api/students/search?name=Asha&page=1  -> @RequestParam
// POST with JSON body                    -> @RequestBody
// Header / cookie                        -> @RequestHeader / @CookieValue

@GetMapping("/search")
public List<Student> search(@RequestParam String name,
                            @RequestParam(defaultValue = "0") int page) {
    return service.search(name, page);
}

@PostMapping
public ResponseEntity<Student> create(@RequestBody Student s) {
    Student saved = service.save(s);
    URI location = URI.create("/api/students/" + saved.getId());
    return ResponseEntity.created(location).body(saved);   // 201 Created
}

@GetMapping("/{id}")
public ResponseEntity<Student> one(@PathVariable Long id) {
    return service.find(id)
            .map(ResponseEntity::ok)                       // 200
            .orElse(ResponseEntity.notFound().build());    // 404
}
// Status codes: 200 OK, 201 Created, 204 No Content, 400 Bad Request,
//               401 Unauthorized, 403 Forbidden, 404 Not Found, 500 Server Error
```

**REMEMBER:** Path = which one. Query param = filter. Body = data. ResponseEntity = custom status.

## 19. DTOs and Records

Never send your database entity straight to the outside world. A **DTO** (Data Transfer Object) is a **custom parcel**: only the fields the client should see or send (hide passwords, avoid loops). Java **records** make DTOs one line.

```
// Entity (database) has everything
@Entity
class User { Long id; String name; String email; String password; /*...*/ }

// DTOs: what goes in and out
record CreateUserRequest(String name, String email, String password) { }
record UserResponse(Long id, String name, String email) { }    // no password!

@Service
class UserService {
    UserResponse create(CreateUserRequest req) {
        User u = new User();
        u.setName(req.name());
        u.setEmail(req.email());
        u.setPassword(encoder.encode(req.password()));
        User saved = repo.save(u);
        return new UserResponse(saved.getId(), saved.getName(), saved.getEmail());
    }
}
// Tools to avoid manual copying: MapStruct or ModelMapper
```

**REMEMBER:** Entity stays inside. DTOs cross the border.

## 20. Validation

A **security gate** that checks input before it enters. Add **spring-boot-starter-validation**, put rules on fields, and trigger them with **@Valid**.

```
public record RegisterRequest(
    @NotBlank(message = "Name is required")
    String name,

    @Email(message = "Invalid email")
    @NotBlank String email,

    @Size(min = 8, message = "Password must be 8+ characters")
    String password,

    @Min(18) @Max(100) int age
) { }

@PostMapping("/register")
public UserResponse register(@Valid @RequestBody RegisterRequest req) {
    return service.create(req);          // runs only if validation passes
}
// Failure -> Spring throws MethodArgumentNotValidException (400).
// Other annotations: @NotNull @NotEmpty @Positive @Past @Future @Pattern(regexp="...")
// Validate path/query params with @Validated on the class + @Min on the parameter.
```

**REMEMBER:** Rules on fields + @Valid on the parameter = automatic 400 for bad input.

## 21. Exception Handling

Instead of ugly stack traces, return **clean, consistent error messages**. **@RestControllerAdvice** is one **complaints desk** that handles errors from every controller.

```
public class ResourceNotFoundException extends RuntimeException {
    public ResourceNotFoundException(String msg) { super(msg); }
}

// In the service:
// repo.findById(id).orElseThrow(() ->
//         new ResourceNotFoundException("Student " + id + " not found"));

record ApiError(int status, String message, LocalDateTime time) { }

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public ApiError notFound(ResourceNotFoundException ex) {
        return new ApiError(404, ex.getMessage(), LocalDateTime.now());
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, String>> invalid(MethodArgumentNotValidException ex) {
        Map<String, String> errors = new HashMap<>();
        ex.getBindingResult().getFieldErrors()
          .forEach(f -> errors.put(f.getField(), f.getDefaultMessage()));
        return ResponseEntity.badRequest().body(errors);
    }

    @ExceptionHandler(Exception.class)                  // last safety net
    @ResponseStatus(HttpStatus.INTERNAL_SERVER_ERROR)
    public ApiError other(Exception ex) {
        return new ApiError(500, "Something went wrong", LocalDateTime.now());
    }
}
```

**REMEMBER:** Throw custom exceptions; one @RestControllerAdvice turns them into clean JSON.

## 22. Layered Architecture

Each layer has **one job**, like departments in a company. Calls go only downward.

```
Controller   -> handles HTTP only: read request, call service, return response
   |
Service      -> business rules, transactions, mapping   (@Service, @Transactional)
   |
Repository   -> talks to the database                   (@Repository)
   |
Database

// Rules of thumb
// 1. Controllers contain NO business logic and NO database code.
// 2. Services never touch HttpServletRequest or ResponseEntity.
// 3. Repositories contain only data access.
// 4. Use interface + implementation for services if you expect many versions:
interface PaymentService { void pay(Order o); }
@Service class UpiPaymentService implements PaymentService { /*...*/ }

// Package layout (by layer):  controller / service / repository / model / dto / exception
// or by feature (bigger apps): student/  course/  payment/  each with its own layers
```

**REMEMBER:** Controller -> Service -> Repository. Each layer one job.

## 23. CORS and API Documentation (Swagger)

**CORS**: browsers block a React app on one address from calling an API on another unless the API allows it - a guest list at the door. **Swagger/OpenAPI** auto-creates a **test page** for your API.

```
// Allow your React app (localhost:5173) to call the API
@Configuration
public class WebConfig implements WebMvcConfigurer {
    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/api/**")
                .allowedOrigins("http://localhost:5173")
                .allowedMethods("GET", "POST", "PUT", "DELETE")
                .allowedHeaders("*");
    }
}
// Or on one controller:  @CrossOrigin(origins = "http://localhost:5173")

<!-- Swagger UI: add springdoc, then open /swagger-ui/index.html -->
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>(latest from springdoc.org)</version>
</dependency>

@Operation(summary = "Get a student by id")       // optional descriptions
@GetMapping("/{id}") public Student one(@PathVariable Long id) { /*...*/ }
```

**REMEMBER:** CORS = who may call my API. Swagger = free interactive docs.

## 24. Calling Other APIs: RestClient and WebClient

Your backend can also be a **customer** of other APIs. **RestClient** (modern, simple, blocking) replaces the old RestTemplate. **WebClient** is non-blocking (reactive).

```
@Service
public class WeatherClient {
    private final RestClient client = RestClient.builder()
            .baseUrl("https://api.example.com")
            .build();

    public WeatherDto today(String city) {
        return client.get()
                .uri("/weather?city={city}", city)
                .retrieve()                              // throws on 4xx/5xx
                .body(WeatherDto.class);                 // JSON -> object
    }

    public void create(PostDto dto) {
        client.post().uri("/posts").body(dto).retrieve().toBodilessEntity();
    }
}
record WeatherDto(String city, double temp) { }

// Older code: RestTemplate.getForObject(url, WeatherDto.class)
// Reactive:   WebClient (spring-boot-starter-webflux) -> returns Mono / Flux
// Declarative: @HttpExchange interfaces (Spring 6) or OpenFeign (Spring Cloud)
```

**REMEMBER:** RestClient = easy outgoing HTTP calls. RestTemplate is the old way.

# PART 4 - DATABASES WITH SPRING DATA JPA

## 25. JPA, Hibernate and Entities

**JPA** is the rulebook, **Hibernate** is the worker who follows it. This is **ORM**: each **table = a Java class**, each **row = an object**. You write Java, Hibernate writes SQL. Boot 3+ uses the **jakarta.persistence** package.

```
@Entity                                 // this class is a table
@Table(name = "students")
public class Student {

    @Id                                 // primary key
    @GeneratedValue(strategy = GenerationType.IDENTITY)   // auto-increment
    private Long id;

    @Column(nullable = false, length = 50)
    private String name;

    @Column(unique = true)
    private String email;

    private int age;                    // column "age" automatically

    @Enumerated(EnumType.STRING)        // store the text, not the number
    private Status status;

    @Transient                          // NOT stored in the database
    private String temp;

    protected Student() { }             // JPA needs a no-arg constructor
    // getters and setters (or Lombok @Getter @Setter)
}
enum Status { ACTIVE, INACTIVE }
```

**REMEMBER:** @Entity = table, @Id = primary key, fields = columns.

## 26. Spring Data JPA Repositories

Write an **interface**, get a full database toolkit - **no implementation code**. Spring writes it at startup, like ordering from a menu instead of cooking.

```
public interface StudentRepository extends JpaRepository<Student, Long> {
    // Student = entity type, Long = type of the id
}

// Free methods: save, saveAll, findById, findAll, existsById,
//               count, deleteById, delete, flush ...

@Service
class StudentService {
    private final StudentRepository repo;
    StudentService(StudentRepository repo) { this.repo = repo; }

    Student create(Student s)   { return repo.save(s); }       // insert or update
    List<Student> all()         { return repo.findAll(); }
    Student get(Long id) {
        return repo.findById(id)                               // Optional
                .orElseThrow(() -> new ResourceNotFoundException("Not found " + id));
    }
    void remove(Long id)        { repo.deleteById(id); }
}

# application.properties (MySQL example)
# spring.datasource.url=jdbc:mysql://localhost:3306/school
# spring.jpa.hibernate.ddl-auto=update      (none / validate in production!)
```

**REMEMBER:** extends JpaRepository = ready-made CRUD.

## 27. Derived Queries, @Query, Paging and Sorting

Method **names become queries** (findBy + FieldName). For complex ones write your own with **@Query**. Paging stops you loading a million rows at once.

```
public interface StudentRepository extends JpaRepository<Student, Long> {

    List<Student> findByName(String name);
    List<Student> findByAgeGreaterThan(int age);
    List<Student> findByNameContainingIgnoreCase(String part);
    Optional<Student> findByEmail(String email);
    List<Student> findByStatusAndAgeBetween(Status s, int from, int to);
    boolean existsByEmail(String email);
    long countByStatus(Status s);
    void deleteByStatus(Status s);

    // JPQL: uses class and field names, not table names
    @Query("SELECT s FROM Student s WHERE s.age > :age ORDER BY s.name")
    List<Student> olderThan(@Param("age") int age);

    // Native SQL
    @Query(value = "SELECT * FROM students WHERE email LIKE CONCAT('%', :d)",
           nativeQuery = true)
    List<Student> byDomain(@Param("d") String domain);

    @Modifying @Transactional
    @Query("UPDATE Student s SET s.status = :st WHERE s.age > :age")
    int updateStatus(@Param("st") Status st, @Param("age") int age);
}

// Paging + sorting
Page<Student> page = repo.findAll(
        PageRequest.of(0, 10, Sort.by("name").ascending()));   // page 0, 10 rows
page.getContent();  page.getTotalPages();  page.getTotalElements();
```

**REMEMBER:** findBy + Field = free query. Use Pageable for big tables.

## 28. Entity Relationships

Real data is connected. @**ManyToOne**: many students in ONE school. @**OneToMany**: one school has MANY students. @**OneToOne**: person-passport. @**ManyToMany**: students-courses (needs a join table). The side with @**JoinColumn** owns the foreign key.

```
@Entity
class School {
    @Id @GeneratedValue private Long id;
    private String name;

    @OneToMany(mappedBy = "school", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Student> students = new ArrayList<>();
}

@Entity
class Student {
    @Id @GeneratedValue private Long id;
    private String name;

    @ManyToOne(fetch = FetchType.LAZY)         // owning side: has the foreign key
    @JoinColumn(name = "school_id")
    private School school;

    @ManyToMany
    @JoinTable(name = "student_course",
        joinColumns = @JoinColumn(name = "student_id"),
        inverseJoinColumns = @JoinColumn(name = "course_id"))
    private Set<Course> courses = new HashSet<>();
}

// Tips
//  - Always set fetch = LAZY for collections and @ManyToOne
//  - Return DTOs, not entities, to avoid infinite JSON loops
//    (School -> students -> school -> students ...)
//  - cascade = ALL: saving/deleting the parent also saves/deletes children
```

**REMEMBER:** mappedBy = the other side owns it. Lazy by default, return DTOs.

## 29. Transactions: @Transactional

A **bank transfer**: money must leave account A AND reach account B - or **neither** happens. A transaction = all steps succeed together, or all are undone (**rollback**). ACID: Atomic, Consistent, Isolated, Durable.

```
@Service
public class BankService {
    private final AccountRepository repo;
    BankService(AccountRepository repo) { this.repo = repo; }

    @Transactional                        // all-or-nothing
    public void transfer(Long fromId, Long toId, double amount) {
        Account from = repo.findById(fromId).orElseThrow();
        Account to   = repo.findById(toId).orElseThrow();

        from.setBalance(from.getBalance() - amount);
        if (from.getBalance() < 0)
            throw new IllegalStateException("Insufficient funds");   // ROLLBACK

        to.setBalance(to.getBalance() + amount);
        // no save() needed: managed entities are saved automatically at commit
    }

    @Transactional(readOnly = true)      // faster for reads
    public List<Account> all() { return repo.findAll(); }
}
// Default: rolls back on RuntimeException (unchecked) only.
//   For checked exceptions:  @Transactional(rollbackFor = Exception.class)
// Gotcha: calling a @Transactional method from the SAME class bypasses the proxy
//         and the transaction does NOT start.
```

**REMEMBER:** @Transactional on the service method: any error undoes everything.

## 30. Databases, Schema and Migrations

For learning use **H2** (in-memory, zero install). For real apps use **MySQL/PostgreSQL**. Don't let Hibernate change a production schema: use **Flyway** or **Liquibase** migrations (version control for your database).

```
# H2 for practice  (add dependency com.h2database:h2)
spring.datasource.url=jdbc:h2:mem:testdb
spring.h2.console.enabled=true          # browse at /h2-console

# ddl-auto options
#   create       drop and rebuild every start  (practice only)
#   create-drop  also drop on shutdown
#   update       add new columns/tables (dev only)
#   validate     only check the schema matches  (production)
#   none         do nothing

-- Flyway: src/main/resources/db/migration/V1__create_students.sql
CREATE TABLE students (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE
);
-- V2__add_age.sql
ALTER TABLE students ADD COLUMN age INT;
-- Flyway runs each file once, in version order, and records it.
```

**REMEMBER:** H2 to learn, MySQL/Postgres to ship, Flyway to change the schema safely.

## 31. The N+1 Problem and Lazy Loading

Loading 100 schools then asking for each one's students fires **1 + 100 queries**, like going to the shop 100 times for one item each. Fix: fetch together in one trip.

```
// BAD: 1 query for schools + N queries (one per school's students)
List<School> schools = schoolRepo.findAll();
schools.forEach(s -> System.out.println(s.getStudents().size()));

// FIX 1: JOIN FETCH
@Query("SELECT s FROM School s JOIN FETCH s.students")
List<School> findAllWithStudents();

// FIX 2: EntityGraph
@EntityGraph(attributePaths = "students")
List<School> findAll();

// FIX 3: return only what you need with a projection / DTO query
@Query("SELECT new com.example.dto.SchoolCount(s.name, COUNT(st)) " +
       "FROM School s LEFT JOIN s.students st GROUP BY s.name")
List<SchoolCount> counts();

// LazyInitializationException = you touched a lazy field after the
// transaction closed. Fetch what you need inside the service.
// See the SQL:  spring.jpa.show-sql=true   (and look for repeated queries)
```

**REMEMBER:** See repeated SQL in logs? Use JOIN FETCH, @EntityGraph or DTO queries.

# PART 5 - SECURITY, TESTING AND PRODUCTION

## 32. Spring Security Basics

Spring Security is the **security guard** of your app. Every request passes through a **chain of filters** (checkpoints). Two ideas: **Authentication** = who are you? (login). **Authorization** = what may you do? (roles). Just adding the starter locks everything.

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity                        // allows @PreAuthorize
public class SecurityConfig {

    @Bean
    SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
          .csrf(csrf -> csrf.disable())     // OK for stateless token APIs
          .authorizeHttpRequests(auth -> auth
              .requestMatchers("/api/auth/**", "/swagger-ui/**").permitAll()
              .requestMatchers(HttpMethod.GET, "/api/products/**").permitAll()
              .requestMatchers("/api/admin/**").hasRole("ADMIN")
              .anyRequest().authenticated())
          .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
          .httpBasic(Customizer.withDefaults());
        return http.build();
    }

    @Bean
    PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); }
}
// Default login (no config): user = "user", password printed in the console.
// Old WebSecurityConfigurerAdapter is gone - use SecurityFilterChain beans.
```

**REMEMBER:** Authentication = who, Authorization = what. Define a SecurityFilterChain bean.

## 33. Users from the Database: UserDetailsService

Teach Spring Security how to **find your users** and **check passwords** (always stored hashed, never plain text).

```
@Service
public class MyUserDetailsService implements UserDetailsService {
    private final UserRepository repo;
    public MyUserDetailsService(UserRepository repo) { this.repo = repo; }

    @Override
    public UserDetails loadUserByUsername(String username) {
        AppUser u = repo.findByUsername(username)
            .orElseThrow(() -> new UsernameNotFoundException("No user " + username));

        return User.withUsername(u.getUsername())
                   .password(u.getPassword())            // already BCrypt-hashed
                   .roles(u.getRole())                   // "USER" -> ROLE_USER
                   .build();
    }
}

// Registering a user: HASH the password before saving
// user.setPassword(passwordEncoder.encode(rawPassword));

// Method-level security
@PreAuthorize("hasRole('ADMIN')")
public void deleteUser(Long id) { /* ... */ }

// Who is logged in?
// SecurityContextHolder.getContext().getAuthentication().getName()
// or in a controller:  public String me(Principal p) { return p.getName(); }
```

**REMEMBER:** Hash with BCrypt. Spring calls loadUserByUsername to verify logins.

## 34. JWT Authentication Flow

A **JWT** is a **wristband at a concert**: you show your ticket once (login), receive a signed wristband (token), and show only that for every later entry. The server needs no session memory. Token = header.payload.signature.

```
1. POST /api/auth/login  {username, password}
2. Server checks password -> creates a signed JWT -> returns {"token": "..."}
3. Client stores it and sends it on every request:
        Authorization: Bearer <token>
4. A filter reads the header, verifies the signature + expiry, sets the user
5. Controller runs. Invalid/expired token -> 401 Unauthorized

// Creating / reading a token (jjwt library, 0.12.x API)
@Service
public class JwtService {
    private final SecretKey key = Keys.hmacShaKeyFor(
            "change-me-use-a-long-random-secret-at-least-32-bytes!".getBytes());

    public String generate(String username) {
        return Jwts.builder()
                .subject(username)
                .issuedAt(new Date())
                .expiration(new Date(System.currentTimeMillis() + 3600_000)) // 1 hour
                .signWith(key)
                .compact();
    }
    public String username(String token) {
        return Jwts.parser().verifyWith(key).build()
                   .parseSignedClaims(token).getPayload().getSubject();
    }
}
```

**REMEMBER:** Login once -> get token -> send Bearer token -> filter verifies it.

## 35. JWT Filter

The filter is the **ticket checker at the gate**: it runs once per request, reads the token and tells Spring Security who the user is.

```
@Component
public class JwtAuthFilter extends OncePerRequestFilter {
    private final JwtService jwt;
    private final UserDetailsService users;
    public JwtAuthFilter(JwtService jwt, UserDetailsService users) {
        this.jwt = jwt; this.users = users;
    }

    @Override
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res,
                                    FilterChain chain) throws ServletException, IOException {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            try {
                String username = jwt.username(header.substring(7));
                UserDetails user = users.loadUserByUsername(username);
                var auth = new UsernamePasswordAuthenticationToken(
                        user, null, user.getAuthorities());
                SecurityContextHolder.getContext().setAuthentication(auth);
            } catch (Exception e) { /* bad token: stay anonymous -> 401 later */ }
        }
        chain.doFilter(req, res);
    }
}
// Register it in SecurityConfig:
// http.addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);
// Never put secrets in the token payload: it is encoded, NOT encrypted.
```

**REMEMBER:** Filter reads Bearer token -> sets Authentication -> request continues.

## 36. Testing Spring Boot Apps

**Pyramid**: many fast unit tests at the bottom, fewer slice tests, very few full-app tests at the top. **JUnit 5** runs tests, **Mockito** creates fake dependencies, **MockMvc** calls controllers without a real server.

```
// 1. UNIT test: no Spring, just Mockito (fastest)
@ExtendWith(MockitoExtension.class)
class StudentServiceTest {
    @Mock StudentRepository repo;
    @InjectMocks StudentService service;

    @Test
    void findsStudent() {
        when(repo.findById(1L)).thenReturn(Optional.of(new Student("Asha")));
        assertEquals("Asha", service.get(1L).getName());
        verify(repo).findById(1L);
    }
}

// 2. CONTROLLER slice: only the web layer
@WebMvcTest(StudentController.class)
class StudentControllerTest {
    @Autowired MockMvc mvc;
    @MockitoBean StudentService service;      // @MockBean in older Boot 3.x

    @Test
    void returnsOk() throws Exception {
        when(service.findAll()).thenReturn(List.of());
        mvc.perform(get("/api/students"))
           .andExpect(status().isOk())
           .andExpect(jsonPath("$.length()").value(0));
    }
}

// 3. REPOSITORY slice:  @DataJpaTest  (in-memory DB, rolls back each test)
// 4. FULL app:          @SpringBootTest  (slow - use sparingly)
```

**REMEMBER:** Unit (Mockito) > @WebMvcTest > @DataJpaTest > @SpringBootTest.

## 37. Caching

Remember answers to **slow or repeated questions**, like writing the answer on a sticky note instead of recalculating. Enable once, then annotate methods.

```
@SpringBootApplication
@EnableCaching
public class App { }

@Service
public class ProductService {

    @Cacheable(value = "products", key = "#id")        // 1st call: runs method
    public Product get(Long id) {                      // later calls: from cache
        System.out.println("Hitting database...");
        return repo.findById(id).orElseThrow();
    }

    @CachePut(value = "products", key = "#p.id")       // update the cache too
    public Product update(Product p) { return repo.save(p); }

    @CacheEvict(value = "products", key = "#id")       // remove stale entry
    public void delete(Long id) { repo.deleteById(id); }
}
// Default cache = in memory. For many servers use Redis:
//   spring-boot-starter-data-redis + spring.cache.type=redis
// Same-class calls skip the cache (proxy rule, like @Transactional).
```

**REMEMBER:** @Cacheable reads, @CachePut updates, @CacheEvict removes.

## 38. Scheduling and Async

**@Scheduled** is an **alarm clock** for methods. **@Async** says 'do this in the background, don't make the user wait'.

```
@SpringBootApplication
@EnableScheduling
@EnableAsync
public class App { }

@Component
class Jobs {
    @Scheduled(fixedRate = 5000)                    // every 5 seconds
    void heartbeat() { System.out.println("alive"); }

    @Scheduled(cron = "0 0 9 * * MON-FRI")          // 9:00 AM on weekdays
    void dailyReport() { /* send report */ }
    // cron = second minute hour day month weekday
}

@Service
class MailService {
    @Async                                          // runs in another thread
    public CompletableFuture<Void> sendWelcome(String to) {
        // slow work...
        return CompletableFuture.completedFuture(null);
    }
}
// Call @Async from ANOTHER bean (proxy rule). In multi-server setups use
// ShedLock so a scheduled job runs only once.
```

**REMEMBER:** @Scheduled = timed jobs, @Async = background work. Enable both first.

# 39. Microservices and Spring Cloud

A **monolith** is one big building. **Microservices** are a village of small independent houses (users, orders, payments), each with its own database, talking over HTTP or messages. Spring Cloud supplies the roads and signboards.

```
                  +----------------+
   Client -------> |  API Gateway   |   single entry, routing, auth
                  +-------+--------+
           +--------------+---------------+
           v              v               v
      user-service   order-service   payment-service   (each: own DB)
           \_______ Eureka (service registry: who lives where) ______/
           Config Server  = central settings   |   Resilience4j = circuit breaker

# Spring Cloud Gateway route
spring.cloud.gateway.routes[0].id=orders
spring.cloud.gateway.routes[0].uri=lb://ORDER-SERVICE      # lb = via Eureka
spring.cloud.gateway.routes[0].predicates[0]=Path=/orders/**

// Call another service declaratively (OpenFeign)
@FeignClient(name = "user-service")
interface UserClient { @GetMapping("/users/{id}") UserDto get(@PathVariable Long id); }

// Circuit breaker: stop calling a broken service, return a fallback
@CircuitBreaker(name = "users", fallbackMethod = "fallback")
UserDto load(Long id) { return userClient.get(id); }
UserDto fallback(Long id, Throwable t) { return new UserDto(id, "unknown"); }
// Start with a monolith. Split only when you truly need to.
```

**REMEMBER:** Gateway + Eureka + Config + circuit breaker. Don't start with microservices.

## 40. Messaging with Kafka (and RabbitMQ)

Services talk through a **post office** instead of phoning each other. A **producer** posts a message to a **topic**; **consumers** collect it whenever ready. Fast, decoupled, survives a service being down.

```
# application.properties
spring.kafka.bootstrap-servers=localhost:9092
spring.kafka.consumer.group-id=email-group
spring.kafka.consumer.auto-offset-reset=earliest

// Producer
@Service
class OrderProducer {
    private final KafkaTemplate<String, String> kafka;
    OrderProducer(KafkaTemplate<String, String> kafka) { this.kafka = kafka; }

    void orderPlaced(String json) { kafka.send("orders", json); }   // topic, message
}

// Consumer (another service)
@Component
class EmailConsumer {
    @KafkaListener(topics = "orders", groupId = "email-group")
    void handle(String message) {
        System.out.println("Send confirmation for: " + message);
    }
}
// RabbitMQ: @RabbitListener + RabbitTemplate (queues, good for task distribution).
// Kafka: huge streams, replayable logs.
```

**REMEMBER:** Producer sends to a topic, consumer listens. Services stay independent.

## 41. Docker and Deployment

Package the app and everything it needs in a **container** (a lunchbox that works the same on every computer), then run it anywhere.

```
# Dockerfile
FROM eclipse-temurin:21-jre
WORKDIR /app
COPY target/demo-0.0.1-SNAPSHOT.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]

# Build and run
./mvnw clean package
docker build -t demo-app .
docker run -p 8080:8080 -e SPRING_PROFILES_ACTIVE=prod demo-app

# docker-compose.yml: app + database together
services:
  db:
    image: mysql:8
    environment:
      MYSQL_ROOT_PASSWORD: secret
      MYSQL_DATABASE: school
  app:
    build: .
    ports: ["8080:8080"]
    environment:
      SPRING_DATASOURCE_URL: jdbc:mysql://db:3306/school
      SPRING_DATASOURCE_PASSWORD: secret
    depends_on: [db]

# Shortcut with no Dockerfile:  ./mvnw spring-boot:build-image
# Hosting: AWS (EC2/ECS/Elastic Beanstalk), Render, Railway, Azure, GCP, Kubernetes
```

**REMEMBER:** mvn package -> Dockerfile -> docker run. Settings come from environment variables.

## 42. Best Practices

Habits of professional Spring developers.

```
// 1. Constructor injection, final fields; Lombok @RequiredArgsConstructor helps
// 2. Thin controllers, fat services (business logic in services)
// 3. DTOs in and out; never expose entities or passwords
// 4. Validate every input (@Valid) and handle errors globally
// 5. Meaningful HTTP status codes and nouns in URLs:
//        GET /api/students/5     (not /getStudent?id=5)
// 6. Secrets in environment variables, never in Git
// 7. ddl-auto=validate + Flyway in production
// 8. Pagination on every list endpoint
// 9. Use @Transactional on service methods; readOnly for queries
// 10. Write tests; keep services stateless
// 11. Version your API: /api/v1/students
// 12. Log with SLF4J, never print passwords/tokens
// 13. Use Actuator health checks + metrics
// 14. Keep dependencies updated (security patches)
```

**REMEMBER:** DTOs, validation, global errors, pagination, secrets outside code.

## 43. Common Mistakes to Avoid

Almost everyone hits these. Recognize them early.

```
// 1. Main class outside the root package -> beans not found (component scan)
// 2. Forgetting @Component/@Service -> NoSuchBeanDefinitionException
// 3. Field injection / circular dependencies (A needs B, B needs A)
// 4. Returning entities directly -> infinite JSON loop + leaked fields
// 5. N+1 queries and EAGER fetching everywhere
// 6. @Transactional on private methods or same-class calls -> does nothing
// 7. LazyInitializationException (touching lazy data after the transaction)
// 8. Storing plain-text passwords (always BCrypt)
// 9. ddl-auto=create in production -> deletes your data!
// 10. Catching Exception and ignoring it
// 11. Storing user data in singleton fields (shared by everyone!)
// 12. Port already in use: server.port=8081
// 13. Jakarta vs javax: Boot 3+ uses jakarta.* imports
// 14. CORS errors: configure allowed origins, don't just disable security
```

**REMEMBER:** Check packages, annotations, lazy loading, passwords, and ddl-auto.

## FINAL PROJECT - Student REST API (entity, repository, service, controller, validation, errors)

Create a project on start.spring.io with Spring Web, Spring Data JPA, Validation and H2. Add these classes under your base package, run it, and test with Postman or Swagger.

```
// ---------- model/Student.java ----------
@Entity
public class Student {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @NotBlank private String name;
    @Email @Column(unique = true) private String email;
    @Min(5) private int age;
    // getters + setters (or Lombok @Data) and a no-arg constructor
}

// ---------- repository/StudentRepository.java ----------
public interface StudentRepository extends JpaRepository<Student, Long> {
    boolean existsByEmail(String email);
    Page<Student> findByNameContainingIgnoreCase(String name, Pageable p);
}

// ---------- exception ----------
public class NotFoundException extends RuntimeException {
    public NotFoundException(String m) { super(m); }
}
@RestControllerAdvice
class ApiErrors {
    @ExceptionHandler(NotFoundException.class)
    ResponseEntity<Map<String,String>> nf(NotFoundException e) {
        return ResponseEntity.status(404).body(Map.of("error", e.getMessage()));
    }
    @ExceptionHandler(MethodArgumentNotValidException.class)
    ResponseEntity<Map<String,String>> bad(MethodArgumentNotValidException e) {
        Map<String,String> m = new HashMap<>();
        e.getBindingResult().getFieldErrors()
         .forEach(f -> m.put(f.getField(), f.getDefaultMessage()));
        return ResponseEntity.badRequest().body(m);
    }
}
```

```
// ---------- service/StudentService.java ----------
@Service
public class StudentService {
    private final StudentRepository repo;
    public StudentService(StudentRepository repo) { this.repo = repo; }

    public Page<Student> search(String name, Pageable p) {
        return repo.findByNameContainingIgnoreCase(name, p);
    }
    public Student get(Long id) {
        return repo.findById(id).orElseThrow(() -> new NotFoundException("Student " + id));
    }
    @Transactional
    public Student create(Student s) {
        if (repo.existsByEmail(s.getEmail()))
            throw new IllegalArgumentException("Email already used");
        return repo.save(s);
    }
    @Transactional
    public Student update(Long id, Student in) {
        Student s = get(id);
        s.setName(in.getName()); s.setEmail(in.getEmail()); s.setAge(in.getAge());
        return s;                                   // saved automatically at commit
    }
    @Transactional
    public void delete(Long id) { repo.delete(get(id)); }
}
```

```
// ---------- controller/StudentController.java ----------
@RestController
@RequestMapping("/api/v1/students")
public class StudentController {
    private final StudentService service;
    public StudentController(StudentService service) { this.service = service; }

    @GetMapping
    public Page<Student> list(@RequestParam(defaultValue = "") String name,
                              @RequestParam(defaultValue = "0") int page,
                              @RequestParam(defaultValue = "10") int size) {
        return service.search(name, PageRequest.of(page, size, Sort.by("name")));
    }
    @GetMapping("/{id}")
    public Student one(@PathVariable Long id) { return service.get(id); }

    @PostMapping
    public ResponseEntity<Student> create(@Valid @RequestBody Student s) {
        Student saved = service.create(s);
        return ResponseEntity.created(URI.create("/api/v1/students/" + saved.getId()))
                             .body(saved);
    }
    @PutMapping("/{id}")
    public Student update(@PathVariable Long id, @Valid @RequestBody Student s) {
        return service.update(id, s);
    }
    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void delete(@PathVariable Long id) { service.delete(id); }
}

# application.properties
spring.datasource.url=jdbc:h2:mem:school
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.h2.console.enabled=true
```

## ONE-PAGE CHEAT SHEET

```
CORE        @Component @Service @Repository @Controller  = "Spring, manage me"
            @Configuration + @Bean = build beans manually
INJECTION   constructor (best) | @Autowired | @Qualifier("name") | @Primary
SCOPE       singleton (default) | prototype | request | session
LIFECYCLE   @PostConstruct  ...use...  @PreDestroy
CONFIG      application.properties/yml | @Value("${x:default}") | @Profile("dev")
            @ConfigurationProperties(prefix="app")
BOOT        @SpringBootApplication = @Configuration+@EnableAutoConfiguration+@ComponentScan
WEB         @RestController @RequestMapping @GetMapping @PostMapping @PutMapping
            @DeleteMapping @PathVariable @RequestParam @RequestBody ResponseEntity
VALIDATION  @Valid @NotBlank @NotNull @Email @Size @Min @Max @Pattern
ERRORS      @RestControllerAdvice + @ExceptionHandler + @ResponseStatus
JPA         @Entity @Table @Id @GeneratedValue @Column @Enumerated @Transient
RELATIONS   @OneToOne @OneToMany(mappedBy) @ManyToOne+@JoinColumn @ManyToMany+@JoinTable
REPOSITORY  extends JpaRepository<Entity, Id> | findByXxx | @Query | Pageable | Sort
TRANSACTION @Transactional (readOnly=true for reads) - rolls back on RuntimeException
SECURITY    SecurityFilterChain | BCryptPasswordEncoder | @PreAuthorize | JWT filter
TESTING     @Test @Mock @InjectMocks @WebMvcTest @DataJpaTest @SpringBootTest MockMvc
EXTRAS      @Cacheable @Scheduled @Async @EventListener @Aspect @Slf4j
HTTP        200 OK  201 Created  204 No Content  400 Bad Req  401 Unauth
            403 Forbidden  404 Not Found  409 Conflict  500 Server Error
FLOW        Controller -> Service -> Repository -> Database  (DTOs at the borders)
```

## Your 90-Day React Roadmap

| | |
|---|---|
| **Days 1-10** | Core Java refresh, Part 1: IoC, DI, beans, annotations. Build a small console app with @Service beans. |
| **Days 11-30** | Parts 2 and 3: Boot setup, REST controllers, validation, exceptions. Build a Student/Book CRUD API with an in-memory list. |
| **Days 31-50** | Part 4: JPA, repositories, relationships, transactions. Connect MySQL/PostgreSQL and add paging. |
| **Days 51-70** | Part 5: Spring Security + JWT, testing, caching. Add login and roles to your API. |
| **Days 71-90** | Docker, deployment, then microservices basics (Gateway, Eureka, Kafka). Deploy one real project and connect it to your React front end. |

**BIG IDEA:** Spring wires the objects, Boot configures the app, you write the business logic.